

DURHAM
COLLEGE

COSC 1100



Class Exercise



COSC 1100

Class Exercise



Selection: Planning to Validate
Part 1 of 2



Selection: Planning to Validate

Goal of Part 1:

Understand why and how we **validate user input**, while also gaining more experience with the initial step(s) of our simple planning process.

Requirements

- Did you come prepared?
 - Control Structures - Selection videos, hopefully done or almost done!
- Collaboration.
 - You will be encouraged to work together with 1-2 colleagues during this exercise. (i.e. groups of 2-3)
 - Students currently outside of Canada are encouraged to communicate and collaborate via MS Teams.
 - The group will share their planning ideas at the end of this class, which will contribute to your grade.

Small Group Discussion

- In your groups discuss the following points and be prepared to share your findings with the class:
 - Should we expect a user to provide appropriate input?
 - How can we help a user provide appropriate input?
 - Is “appropriate” user input the same as “correct” user input?

Whole Class Discussion

- Why should we validate user input?
 - You have probably already stated an answer in the first two discussion questions
- As a class, lets list some different aspects of user input we might validate.

How to Complete Part 1 of This Class Exercise

- Consider a program that accepts some input about students in COSC 1100. The professor will compile a list of these attributes.
- We want to write a program that will accept only valid values for these attributes, and then output them.
 1. Analyze the problem; imagine and discuss validation rules for each of these variables.
 2. Make a discussion post including your notes on DC Connect (**COSC 1100 -> Activities -> Discussion -> Class Exercises**), including all group members' names.
 3. Be prepared to discuss your notes in class.

Planning to Validate, Continued

- Let's consider the rules we've made, and make a flowchart!

Class Exercise Part 1 Discussion

- Consider the validation rules presented.
 - Were the rules written by other groups much different than your group?
 - Were different students' perspectives useful in starting to think about validation?
 - What constructive ways could the other groups' rules be improved or clarified?
- With the introduction of validation is all this planning starting to seem more worthwhile?

Class Exercise Part 1 Summary

- Hopefully you can see how planning the validation rules in advance is going to inform your code.
- You've got a plan and we will write the code based on it in our next class.

COSC 1100

Class Exercise



Selection: Planning to Validate
Part 2 of 2



Selection: Validating User Input

Goal of Part 2:

Experience with implementing control structures from a flow chart for the purpose of user validation.

Requirements

- Did you come prepared?
 - Control Structures - Selection videos, ideally all done!
- Collaboration.
 - You will be discussing and working together in small groups, hopefully the same as last time (attendance and technology permitting!).
 - Someone will submitting a completed Python code file **for marks** – include all group members' names!

Validation Code Example Using `isnumeric()`

```
# Display message: "Please enter the number of sales for this salesperson: "  
# Attempt to store the user input in variable sale_amount  
sale_amount = input('Please enter the number of sales: '))  
if sale_amount.isnumeric():  
    sale_amount = int(sale_amount)  
    invalid = True  
else:  
    invalid = False  
  
# If sale_amount was not stored successfully  
if not invalid:  
    # display message, "Sorry, the number of sales must be a whole number"  
    print('Sorry, the number of sales must be a positive whole number.')  
else:  
    # Otherwise, carry on with processing/output
```

Validation Code Example Using Try / Except

```
# Display message: "Please enter the gross sales for this salesperson: $"
# Attempt to store the user input in variable gross_sales
try:
    gross_sales = float(input('Please enter the gross sales: $'))
    invalid = True
except:
    invalid = False

# If gross_sales was not stored successfully
if not invalid:
    # display message, "Sorry, the gross sales must be a positive number"
    print('Sorry, the gross sales must be entered as a positive number.')
else:
    # Otherwise, carry on with processing/output
```

Plan to Program!

- With consideration to the plan developed in the previous class, code the application using Python!
 1. Code your solution to the given problem based on the plan provided or your own planning notes.
 2. Ask questions, and make sure all group members know what is happening!
 3. Either have it checked during class,
or submit your group's finished Python code file to DC Connect
(COSC 1100 -> Activities -> Assignments -> Class Exercise: Numeric and String Data), including all group members' names.

This is meant to be done during class time; submissions received after our class time ends may be considered late.

Class Exercise Part 2 Discussion

- Consider the process of implementing user validation; what aspects of this did you find most challenging?
- Did you write comments related to validation as you did this?
Did the comments happen before or after the coding?
Did that make things easier or harder for you?
- Compared to writing pseudo-code, how helpful is the development of a flowchart in the planning process?

Class Exercise Part 2 Summary

- You've implemented some validation! It will *definitely* not be the last time, as behaviours like this will be normal in all of your future assignments.
- You may need to get used to flowcharts, too! We'll revisit that in a couple of weeks, if not before.